ContextWorld: Benchmarking In-Context Dynamics Adaptation in Latent Visual World Models

Abstract

Latent world models predict future states from visual observations and action sequences within a learned representation space. When environment dynamics vary, accurate prediction requires inferring hidden dynamics from interaction history. However, because models can exploit immediate visual shortcuts or regress to average dynamics to achieve low loss without history conditioning, conventional prediction accuracy fails to reveal genuine adaptation. We introduce ContextWorld, a benchmark designed to evaluate in-context dynamics adaptation in action-conditioned latent world models under strictly frozen weights. ContextWorld employs paired counterfactual evaluations—varying interaction histories while holding the current observation and future action sequence fixed—to concurrently quantify adaptation accuracy (whether predictions correctly distinguish between diverging future states) and history sensitivity (whether matching history yields lower prediction error than mismatched history). Spanning nine tasks across four environments, the benchmark provides separate training and evaluation splits with continuous interaction rollouts under differing dynamics. Our systematic evaluation of representative latent world models (LeWM, PLDM, and DINO-WM) yields three key findings: First, models trained under fixed dynamics make negligible use of interaction history across most tasks. Second, mixing multi-dynamics data improves history responsiveness, but the benefits of data scaling and the relative effectiveness of training strategies vary substantially across tasks. Third, under certain training schemes, models fail to effectively leverage history even on the training distribution, despite steadily decreasing training loss. These results reveal a notable mismatch between prediction-loss reduction and in-context dynamics adaptation, suggesting that standard prediction objectives do not reliably drive models to use interaction history and highlighting the need for objectives and benchmarks explicitly targeted at context-driven physical reasoning.

1. Introduction

Visual world models learn environment dynamics to predict future states [19], with latent world models performing these predictions within learned representations of visual observations [20, 21]. A model trained under fixed dynamics can achieve high prediction accuracy without utilizing interaction history to adapt its predictions to current conditions. However, such predictions can fail when environment dynamics vary [33]. For example, visually identical objects can move different distances under the same push if surface friction differs. Interaction history provides evidence about these hidden physical properties. We investigate whether latent world models can leverage this contextual evidence to predict future states under varied dynamics *without updating their weights*—an ability we term *in-context dynamics adaptation*.

Physical prediction benchmarks evaluate whether observed interactions help models infer hidden properties or predict downstream outcomes governed by them [4, 32]. However, benchmarks such as Physion++ evaluate visual models through auxiliary readout classifiers [32]. We introduce ContextWorld to evaluate in-context dynamics adaptation directly within action-conditioned latent world models, leveraging their existing latent predictors without training separate classifiers for future events. The benchmark spans nine tasks across four environments, covering physical parameter variations and interaction rules, and provides separate training and evaluation data.

In its paired tasks, two dynamics conditions share the identical current observation and future action sequence, but produce diverging ground-truth futures. Interaction history is therefore strictly necessary to distinguish the two outcomes. Because a deterministic model that ignores history receives identical inputs and yields identical predictions for both conditions, it scores at most 50% on paired future selection when ties count as failures. We evaluate adaptation accuracy by testing whether predictions distinguish the true futures under different dynamics. We assess history sensitivity by comparing prediction errors with matching and mismatched histories.

Using ContextWorld, we benchmark three representative latent world models: LeWM [24], PLDM [30], and DINO-

WM [37]. We find that after training under fixed dynamics, these models make negligible use of interaction history across most tasks. To examine how this adaptation ability can be acquired, we mix ContextWorld training data with original rollouts and compare training from scratch against fine-tuning models pretrained under fixed dynamics. We train models separately for each task; DINO-WM retains its frozen pretrained visual encoder in both settings. We further scale up the training data and examine the impact of freezing the visual encoder in LeWM and PLDM. Our experiments reveal that data scaling improves history utilization on several tasks, while the most effective model and training strategy vary substantially by task. For example, starting from the same pretrained weights, freezing PLDM's visual encoder improves its ability to distinguish future states under varying contact friction, but reduces this ability under motion damping.

To understand why models often fail to leverage history, we analyze model predictions on the training distribution itself. Under certain training schemes, training loss steadily decreases even while models make limited use of interaction history to distinguish dynamics conditions. Because this failure occurs on the training data, it cannot be explained by out-of-distribution generalization alone, suggesting that standard prediction objectives may not sufficiently encourage models to use interaction history for prediction.

Our contributions are:

1. **The ContextWorld benchmark.** We introduce nine hidden-dynamics tasks across four environments, with separate training and evaluation data. Paired counterfactual evaluations measure future discrimination and the benefit of matching history directly from latent predictions, without weight updates or auxiliary readouts.
2. **Systematic empirical comparison.** We benchmark representative latent world models across data scales, training strategies (scratch vs. fine-tuning), and visual encoder freezing schemes, demonstrating that scaling benefits and optimal adaptation strategies depend heavily on the underlying physical regime.
3. **Empirical evidence of adaptation failures during training.** We show that training loss can steadily decrease even when models fail to leverage interaction history on the training data, providing empirical evidence that standard prediction losses do not reliably drive in-context dynamics adaptation.

2. Related Work

Visual world models. Visual world models learn environment dynamics to simulate and predict future states. Early frameworks [19] combined learned visual representations with recurrent dynamics to support internal policy training. Subsequent methods advanced this paradigm through online planning (PlaNet [20]) and policy learning via imagined latent trajectories (Dreamer [21]). While these foundational approaches rely on pixel-level image reconstruction to learn visual representations, recent Joint-Embedding Predictive Architectures (JEPAs) [23] shift the training objective toward predicting target representations directly without reconstructing pixels. This reconstruction-free principle has been applied to masked spatial regions (I-JEPA [2]) and extended to temporal video representation learning (V-JEPA [5]). To support physical interaction and decision-making, models like V-JEPA 2 [3] introduce an action-conditioned predictor trained on robot interactions atop pretrained video representations.

Action-conditioned latent world models also differ in how they learn visual representations. DINO-WM [37] trains a predictive dynamics model on top of strictly frozen pretrained features for planning from offline trajectories. In contrast, PLDM [30] jointly optimizes the visual encoder and the predictor using reward-free offline data. Addressing the inherent instabilities of joint training, LeWM [24] streamlines the objective into a pure latent prediction loss paired with a single representation regularizer.

In-context adaptation. When environment dynamics shift between training and deployment, interaction history provides a critical signal for adaptation without requiring costly weight updates. Early approaches in meta-reinforcement learning [6] leveraged this principle by training agents to adapt from limited experience across tasks. Specifically, RL^2 [14] implicitly stores interaction history within a recurrent network's hidden state, while PEARL [29] explicitly infers a task representation from experience to condition the policy. Paralleling the success of large language models, which learn from in-context examples without parameter updates [8, 13], this adaptation paradigm has increasingly been extended to visual world models.

Within world models, contextual history is utilized to guide predictions of action outcomes under novel conditions. For instance, OSWM [16] pretrains a transformer on synthetic dynamics, using observed state transitions as context to forecast futures in new environments. L2World [33] investigates the mechanics of this adaptation, distinguishing between using context to identify previously seen environments versus learning entirely new dynamics on the fly. It systematically studies the effects of context length and training environment diversity through multi-step prediction errors and context perturbations. For robotic control, ICWM [34] employs task-agnostic interaction history to adapt policies to novel camera viewpoints without weight updates, using ablations over correct, absent, and mismatched interaction contexts to validate history reliance.

Benchmarks. To evaluate adaptation across diverse environments, existing benchmarks introduce controlled variations in dynamics, appearance, or underlying task rules. For instance, Meta-World [35] and Procgen [11] test generalization across manipulation tasks and procedurally generated levels. CausalWorld [1] and CARL [7] directly vary environment properties, while the XLand series (XLand-MiniGrid [26], XLand-100B [27]) evaluates policy adaptation from interaction history across distributions of tasks.

Within the domain of physical prediction, several benchmarks focus on counterfactual reasoning and hidden property inference. CoPhy [4] asks models to predict outcomes following changes to a scene's initial configuration, utilizing the original trajectory as evidence of hidden properties. Filtered-CoPhy [22] refines this by ensuring the observed trajectory contains sufficient information for the counterfactual prediction. ComPhy [9] probes reasoning regarding hidden mass and charge through predictive and counterfactual video querying, while Physion++ [32] tests whether interaction history aids in predicting future contact in paired trials that share the identical current observation but differ in physical properties. While these benchmarks provide critical insights, Physion++ assesses visual models via auxiliary readout classifiers rather than directly scoring their predicted representations [32].

Beyond visual domains, a recent time-series benchmark [36] reveals that low absolute prediction error can misleadingly coexist with incorrect response directions when actions change. A critical remaining question is whether and when modern action-conditioned latent world models learn to leverage interaction history for accurate, in-context physical prediction. ContextWorld addresses this gap by directly evaluating latent predictors through controlled comparisons across models, tasks, data scales, and training strategies.

3. ContextWorld Benchmark

ContextWorld is designed to evaluate in-context dynamics adaptation—specifically, whether models leverage interaction history to accurately predict future states conditioned on an action sequence. This section formalizes the prediction setting, describes how we construct the tasks and datasets, and explains the paired evaluation protocol and metrics.

3.1. Problem Formulation

To strictly isolate in-context adaptation from weight updates, all model parameters θ remain fixed during evaluation. The model receives exclusively visual observations and actions. For a target horizon τ, the model receives an interaction history H_t, comprising visual observations up to and including time t and the actions executed between them, alongside a planned future action sequence $A_{t:t+\tau-1}$. The prediction and its target are formulated as:

$$\begin{aligned} \hat{z}_{t+\tau} &= f_\theta(H_t, A_{t:t+\tau-1}), \\ z_{t+\tau} &= e_\theta(o_{t+\tau}). \end{aligned} \tag{1}$$

Here, $o_{t+\tau}$ is the ground-truth visual observation at time $t+\tau$, and e_θ is the model's native target encoder. The target encoder is also frozen during evaluation. We compare predictions directly with these encoded targets in latent space.

3.2. Task Design

To systematically evaluate dynamics adaptation, we construct nine tasks spanning four base environments by introducing controlled variations in physical parameters and underlying interaction rules. These environments build upon TwoRoom navigation [37], the PushT pushing environment [10], Reacher from the DeepMind Control Suite [31], and Cube from OGBench [28]. The tasks and their corresponding history lengths are summarized in Table 1.

To test whether models use interaction history for prediction, we construct paired samples for six tasks: pusher displacement, arm mass, contact friction, motion damping, gripper carry rule, and portal exit. The paired samples share the current observation and future action sequence, but have different hidden dynamics. For each sample, we generate the history and ground-truth future in one continuous simulation with fixed dynamics. The two futures differ, so accurate prediction requires information from the history (Fig. 1). Construction checks are detailed in Appendix B.

The speed task tests whether models can use history to predict motion at unseen speeds. We hold the current observation and future actions fixed and provide histories recorded at different speeds. Action delay covers several delay values to test predictions before and after an action takes effect. For example, with delays of six or ten simulator steps, no movement occurs within the next five steps. These delays produce the same future state, so we score them as one response group and weight all groups equally (Appendix C). For the door task, we test whether the model correctly predicts passage or blockage from the interaction history.

3.3. Dataset

For each task, we provide Training, Development, and Test splits with no shared interaction trajectories. The Development split is used to select methods and checkpoints. These splits may share dynamics parameter values or interaction rules. The speed task also tests speeds not used in training, both within the training range (interpolation) and outside it (extrapolation).

The paired tasks provide 32,768 training pairs per task, except for gripper carry, which provides 10,000. Each has 256 Development pairs and 256 Test pairs. Appendix D

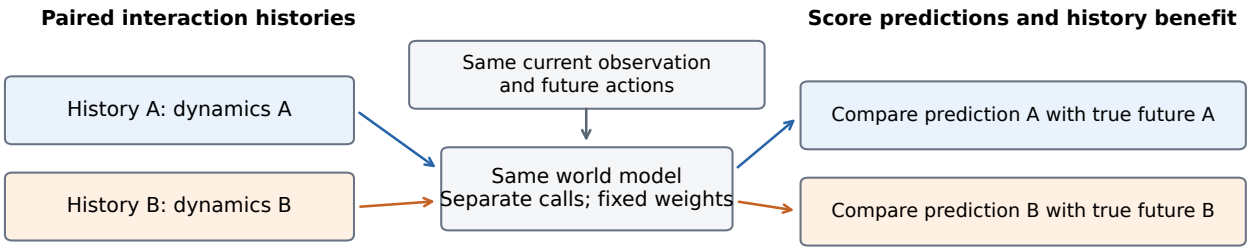


Figure 1. **Controlled prediction and history comparisons.** In the paired tasks, histories from two dynamics conditions end at the same current observation and share future actions, but lead to different true futures. We score predictions against their corresponding targets and then swap the input histories while retaining each target to test whether matching history reduces error. The same model weights are used throughout.

Table 1. The nine tasks test whether interaction history helps models predict the effects of hidden dynamics. History length h counts observation frames, including the current frame; sampling details are given in Appendix A.

Task	Environment	h	What history helps the model predict
Speed	TwoRoom	3	How far the agent moves under the same action
Pusher displacement	PushT	3	Whether the same action moves the pusher a shorter or longer distance
Arm mass	Reacher	3	How the same torque changes arm motion
Action delay	TwoRoom	7	When an action takes effect
Contact friction	PushT	3	How friction changes object motion during contact
Motion damping	PushT	3	How quickly object motion decays after contact ends
Gripper carry rule	Cube	3	Whether a closed gripper carries the cube
Door traversability	TwoRoom	3	Whether a visually identical door permits passage
Portal exit	TwoRoom	3	Where the agent emerges after entering the same portal

provides complete dataset statistics, dynamics settings, and split checks for all nine tasks.

3.4. Evaluation

A model can change its predictions with the interaction history and still predict the wrong future. We therefore evaluate whether predictions distinguish the true futures under different dynamics, measure prediction error, and compare errors with matching and mismatched histories. We report benchmark results on the Test split.

Prediction accuracy. All tasks except Speed and action delay use the same accuracy rule. For these tasks, we compare each prediction with both true futures. A prediction using the history from condition A is counted as correct only if it is closer to A's true future than to B's, and vice versa. For pair i, z_i^A and z_i^B denote the true future representations, and $\hat{z}_i^A$ and $\hat{z}_i^B$ denote the corresponding predictions. Using mean squared error d, we report accuracy as the percentage of correct predictions across both conditions in N pairs:

$$
\begin{aligned}
A_{\text{future}} &= \frac{100}{2N}\sum_{i=1}^{N}(I_i^A + I_i^B), \\
I_i^A &= \mathbf{1}\{d(\hat{z}_i^A, z_i^A) < d(\hat{z}_i^A, z_i^B)\}, \\
I_i^B &= \mathbf{1}\{d(\hat{z}_i^B, z_i^B) < d(\hat{z}_i^B, z_i^A)\}.
\end{aligned}
\tag{2}
$$

Speed and action delay involve more than two dynamics conditions. For each tested speed, we compare predictions from different histories with the true future at that speed. We count the prediction from the matching history as correct only when its error is lower than that of every other prediction. For action delay, the prediction horizon is five simulator steps. Delays 0–4 form five separate response groups, while delays 5–10 form one group because the action has not yet taken effect. We assign each prediction to the group containing the closest true future, compute accuracy for each group, and report the mean across the six groups. Appendix E gives the detailed scoring rules.

Multi-step prediction error. High prediction accuracy does not guarantee low prediction error. We therefore evaluate predictions over multiple steps, using the same future action sequences for all dynamics conditions in each scene. Models roll out from the observed interaction history without receiving further ground-truth observations. We evaluate five successive predictions, up to 25 simulator steps ahead (Appendix E).

We compare prediction errors with a reference that makes the same prediction across dynamics conditions [25]. For each scene, action sequence, and prediction time, this reference is the mean of the true future representations. With equally weighted conditions, this mean gives the lowest squared error among predictions that do not distinguish the conditions [18]. For scene q, let z_{qkct} and $\hat{z}_{qkct}$ denote the true and predicted future representations for dynamics condition k, action sequence c, and prediction time t. The reference prediction is $\bar{z}_{qct} = K_q^{-1} \sum_k z_{qkct}$, where K_q is the number of dynamics conditions. We average the model's squared errors within each scene to obtain L_q, and the reference errors to obtain B_q. Both averages cover all K_q conditions, C_q action sequences, and $T = 5$ prediction times. We then report the ratio of the summed model errors to the summed reference errors:

$$
\begin{aligned}
L_q &= \frac{1}{K_q C_q T} \sum_{k,c,t} \|\hat{z}_{qkct} - z_{qkct}\|^2, \\
B_q &= \frac{1}{K_q C_q T} \sum_{k,c,t} \|z_{qkct} - \bar{z}_{qct}\|^2, \\
E/B &= \frac{\sum_q L_q}{\sum_q B_q}.
\end{aligned}
\tag{3}
$$

Lower E/B is better: 0 means an exact prediction in the target representation, and 1 matches the error of the mean-future reference. Values below 1 outperform this reference; values above 1 indicate larger errors. To measure the benefit of matching history, we also compare errors with matching and mismatched histories.

History controls. To test whether matching history helps prediction, we perturb the input history [33]. For each sample, we replace the history with one from a different dynamics condition in the same scene. The current observation and future actions remain unchanged. We evaluate both predictions against the true future of the original sample. History (%) is the percentage of comparisons in which matching history gives strictly lower prediction error than mismatched history. Speed also uses this strict comparison as its task score.

History (%) measures how often matching history reduces error, but not by how much. We therefore also report the reduction relative to the same reference error used for E/B. Let E_+ and E_- be the summed scene errors with matching and mismatched histories, respectively, using the averaging over conditions, actions, and times in Eq. 3. For each sample, errors from multiple mismatched histories are averaged equally. With $B = \sum_q B_q > 0$, the normalized error reduction is

$$
G_H = \frac{E_- - E_+}{B}. \tag{4}
$$

A positive G_H means that matching history reduces error, but does not imply that the predictions are accurate. Their overall error is measured by E/B.

Conditional response error. Training loss can decrease even when the model does not accurately predict how different dynamics change the future. We therefore compare the difference between the two predicted futures with the difference between the two true futures. For pair i, let $\Delta z_i = z_i^B - z_i^A$ denote the true difference and $\Delta \hat{z}_i = \hat{z}_i^B - \hat{z}_i^A$ the predicted difference. We define normalized response error (NRE) as the summed squared error between these differences, divided by the summed squared magnitude of the true differences:

$$
\mathrm{NRE} = \frac{\sum_i \|\Delta \hat{z}_i - \Delta z_i\|^2}{\sum_i \|\Delta z_i\|^2}. \tag{5}
$$

Lower NRE is better: 0 means that the predicted difference exactly matches the true difference. Predicting no difference gives NRE = 1, although other incorrect predictions can produce the same value. Errors shared by both predictions cancel when we subtract them, so a low NRE does not guarantee that either predicted future is accurate.

Comparing prediction errors. E/B and NRE measure errors in each checkpoint's target representation. Normalization does not make these representations equivalent, so lower values across models do not necessarily indicate more accurate physical predictions. Changes across training checkpoints can also reflect changes in the target encoder. A fixed target representation removes this source of variation.

Reporting results. We report the mean and standard deviation across training runs with the same settings, weighting each run equally. For multi-step errors, we also report confidence intervals that reflect variation across sampled scenes or source trajectories, rather than across training runs (Appendix E).

4. Experiments

4.1. Experimental Setup

To study how training data affect adaptation, we compare training on original-environment data alone with training

on a mixture of original-environment and ContextWorld data. We use the original-environment datasets released with LeWM [24]. For each task, mixed training samples 50% of its trajectory windows from the corresponding original-environment dataset and 50% from the task's ContextWorld Training split.

We evaluate LeWM [24], PLDM [30], and DINO-WM [37] to examine how original-environment pretraining and visual encoder updates affect adaptation. Table 2 defines the four training schemes used in the comparisons. Original and Mixed each train for 10 epochs. The two Original $\rightarrow$ Mixed schemes start from the same Original checkpoint and train for another 10 epochs on mixed data, either updating or freezing the visual encoder. Each mixed-data model is trained for one task, so a row of nine task results represents separate task-specific checkpoints. Optimization and sampling details appear in Appendix G.

We also evaluate planning with the cross-entropy method (CEM) [12] in the original environments to test whether models retain their planning ability after training on mixed data. These scores measure performance in the original environments, not adaptation to hidden dynamics.

4.2. Adaptation Results

Table 3 compares task accuracy on the Test split. With Original training, scores on the seven binary tasks range from 38.48% to 52.21%, close to or below the 50% reference level. Mixed training improves performance on several tasks: for example, arm mass accuracy rises from 50.07% to 85.74% for LeWM and from 51.82% to 91.02% for PLDM. These gains show that training with ContextWorld data can improve task accuracy, although the benefits vary across tasks and models.

The relative performance of the models depends on the task. With Mixed training, LeWM and PLDM reach 97.70% and 100.00% on action delay, compared with 16.65% for DINO-WM. On pusher displacement, the ordering reverses: DINO-WM reaches 97.66%, while LeWM and PLDM score 50.78% and 56.25%. All three models reach at least 99.83% on door traversability. These contrasts show why adaptation should be evaluated across different dynamics rather than summarized by a single model ranking.

Task accuracy alone does not establish how well the model uses history or predicts the complete future state. We therefore examine history controls and prediction errors on Development. For LeWM on pusher displacement, Original $\rightarrow$ Mixed (Frozen Encoder) raises History from 34.05% to 98.05% and reduces NRE from 1.015 to 0.144 relative to Original. Thus, matching history more often improves prediction, and the predicted response to changing dynamics is closer to the true response in the evaluated representations. However, on the extended action sequences used for multi-step evaluation, this model has $E/B = 1.464$; the corresponding PLDM model has $E/B = 1.445$. Both exceed the mean-future reference error. These multi-step queries differ from the one-step task queries: successful discrimination on the latter does not establish accurate prediction over longer action sequences. Appendix H reports the full diagnostics and original-environment CEM results.

4.3. Analysis of Training Effects

Freezing the visual encoder helps some tasks and hurts others, even when the starting weights are the same. For PLDM, Original $\rightarrow$ Mixed (Frozen Encoder) increases contact friction Test accuracy from 50.00% to 73.63% relative to joint training, but reduces motion damping accuracy from 79.88% to 54.49%. Each comparison uses the same task data, mixture, initialization, and training duration. Freezing therefore changes the outcome of training in a task-dependent way; it is not a general solution to weak adaptation. Because the projector and other trainable modules still update, this comparison does not isolate the role of a fixed target representation.

More training data can improve adaptation, but the gain depends on the training scheme. For LeWM on pusher displacement, increasing the training set from 2,048 to 8,192 to 32,768 pairs raises Development accuracy from 55.66% to 82.42% to 90.82% with Original $\rightarrow$ Mixed (Joint); NRE falls from 0.810 to 0.390 to 0.222. In contrast, Mixed training scores 50.00% with 2,048 pairs and 50.78% with 32,768 pairs. The additional data therefore help the pretrained model in this comparison but do not produce a similar gain with direct mixed training. These experiments keep the number of epochs fixed, so increasing the dataset can also increase training updates and change trajectory coverage. They do not isolate the effect of data quantity at a fixed compute budget.

Increasing model capacity alone does not resolve the weak performance in our tested setting. We jointly expand the encoder, representation width, and predictor of LeWM and PLDM from approximately 18M to 183M parameters, using Mixed training on the same 32k friction and damping datasets for 10 epochs. Across the four Development comparisons, task accuracy changes by -8.01 to $+0.20$ percentage points, and original-environment CEM success decreases in all four. This result limits a simple capacity explanation for these runs; it does not rule out gains with different optimization settings or training budgets.

To determine whether weak adaptation arises only at evaluation time, we also score fixed queries from the training data. Figure 2 compares PLDM's Joint and Frozen Encoder schemes on friction and damping, using 512 training pairs and 256 Development pairs per task. In friction joint training, the recorded prediction loss decreases by 18.05% between epochs 5 and 10, with both its mean and median

Table 2. Training schemes. Original data come from the corresponding environment; mixed data use the 50/50 sampling described in the text. LeWM and PLDM start Original and Mixed from random weights. DINO-WM starts with a pretrained, frozen visual encoder and a randomly initialized predictor.

Scheme	Starting weights	Training data	Visual encoder	Other trainable modules
Original	Model initialization	Original only	Updated[†]	Updated
Mixed	Model initialization	Mixed	Updated[†]	Updated
Original → Mixed (Joint)	Original checkpoint	Mixed	Updated	Updated
Original → Mixed (Frozen Encoder)	Same Original checkpoint	Mixed	Frozen	Updated

[†]DINO-WM's visual encoder remains frozen in all experiments, so Original → Mixed (Joint) is not evaluated for this model. In Original → Mixed (Frozen Encoder), only the visual encoder is frozen; the remaining trainable modules, including any projector and action encoder, continue to update. All schemes retain the model's original training losses.

Table 3. Test task accuracy (%, higher is better). Entries report means across available training runs; a second line gives the sample standard deviation where three runs are available. Other entries use one run. Speed reports unseen speeds within the training range; action delay uses seven-frame history. A dash denotes no comparable checkpoint. Each task uses its own checkpoint.

Model	Training scheme	Speed	Pusher displacement	Arm mass	Action delay	Contact friction	Motion damping	Gripper carry rule	Door traversability	Portal exit
LeWM	Original	58.04 ±1.70	48.96 ±1.00	50.07 ±0.11	—	49.93 ±0.23	50.00 ±0.00	52.21 ±1.11	50.33 ±0.00	50.07 ±0.11
LeWM	Mixed	98.19 ±0.57	50.78	85.74	97.70 ±0.59	50.00	50.00	50.00	99.83 ±0.29	50.78
LeWM	Original → Mixed (Joint)	96.89	91.80	87.30	—	50.39	50.00	50.00	99.83	50.39
LeWM	Original → Mixed (Frozen Encoder)	98.44	97.46	77.15	—	86.13	53.91	76.37	100.00	77.93
PLDM	Original	48.41 ±4.33	45.57 ±1.88	51.82 ±1.30	—	50.00 ±0.34	50.26 ±0.30	50.78 ±0.20	50.06 ±0.10	50.07 ±0.11
PLDM	Mixed	96.78 ±2.02	56.25	91.02	100.00 ±0.00	50.00	58.20	49.80	100.00 ±0.00	72.27
PLDM	Original → Mixed (Joint)	98.33	59.18	84.77	—	50.00	79.88	50.20	100.00	52.54
PLDM	Original → Mixed (Frozen Encoder)	96.44	98.63	81.84	—	73.63	54.49	52.93	100.00	91.21
DINO-WM	Original	57.11	38.48	50.78	—	50.39	49.80	50.00	50.00	50.78
DINO-WM	Mixed	59.41 ±3.87	97.66	78.32	16.65 ±0.03	84.18	56.45	60.16	100.00 ±0.00	100.00
DINO-WM	Original → Mixed (Frozen Encoder)	84.78	—	—	—	—	55.27	62.89	—	—

decreasing at every recorded epoch. Yet training-query NRE remains near the zero-response reference, at 1.009 and 1.011. The model therefore fails to recover the conditional response even on these training queries; a shift to the evaluation distribution cannot explain this failure alone.

The positive controls also matter: with the encoder frozen, friction training-query NRE falls from 0.674 to 0.587, while joint training gives the stronger response on damping. Development shows the same ordering between these schemes. Thus, the training data support better conditional responses under some schemes, even when another scheme remains weak on training queries. This does not establish that every aspect of the future state has been learned accurately.

Together, these results show that reducing the training objective is not a sufficient indicator of acquiring the measured adaptation ability. The loss averages prediction errors across mixed data and prediction positions, whereas the response metric tests differences between paired futures. Their behavior can therefore differ, and the learned target representation can also change during training. Our evidence identifies a gap between optimization progress and conditional prediction, but does not attribute that gap to a unique loss term or representation component. The direct training-query evidence is limited to the tested configurations; it is not a common causal explanation for all nine tasks.

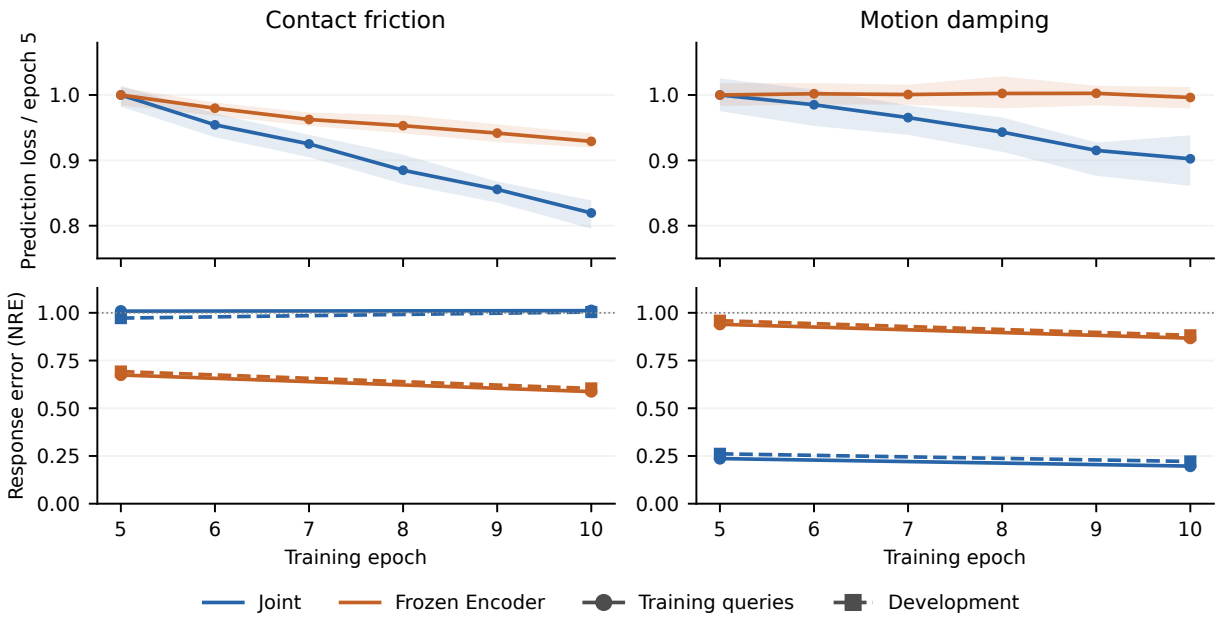


Figure 2. **Lower training loss does not guarantee a more accurate conditional response.** PLDM Joint and Frozen Encoder start from the same original-environment checkpoint within each task. Top: recorded prediction-loss means, normalized by each run's epoch-5 mean; shading shows the batch interquartile range, not a confidence interval. Bottom: NRE on fixed training and Development queries at the two evaluated checkpoints, predicting five simulator steps ahead. Lines connect measured points and do not show intermediate NRE values. NRE = 1 is the zero-response reference. Loss averages mixed training batches; NRE uses paired task queries.

5. Conclusion

ContextWorld evaluates whether latent visual world models use interaction history to predict under hidden dynamics. Its nine tasks combine controlled history comparisons with task accuracy and prediction-error measurements, separating successful discrimination from accurate conditional prediction. Across three model families, training on mixed data improves performance on several tasks, but model advantages and the effects of encoder freezing depend on the task. Data scaling helps some training schemes, while the tested capacity increase does not consistently improve weak tasks. Training-query analysis further shows that prediction loss can decrease without recovering the conditional response. These findings motivate evaluating adaptation directly rather than treating loss reduction as evidence that it has been learned.

The benchmark measures predictions in each model's representation, not a shared physical error space. Most tasks test adaptation among dynamics conditions represented in training; Speed additionally tests unseen values. The results concern task-specific checkpoints, and original-environment planning does not establish planning gains under hidden dynamics. These boundaries define the scope of the comparisons and leave multi-task learning, broader dynamics generalization, and decision benefits for further study.

References

[1] Ossama Ahmed, Frederik Träuble, Anirudh Goyal, Alexander Neitz, Yoshua Bengio, Bernhard Schölkopf, Manuel Wüthrich, and Stefan Bauer. CausalWorld: A Robotic Manipulation Benchmark for Causal Structure and Transfer Learning. In *International Conference on Learning Representations*, 2021. 3

[2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-Supervised Learning From Images With a Joint-Embedding Predictive Architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023. 2

[3] Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. *arXiv preprint arXiv:2506.09985*, 2025. 2

[4] Fabien Baradel, Natalia Neverova, Julien Mille, Greg Mori,

and Christian Wolf. CoPhy: Counterfactual Learning of Physical Dynamics. In *International Conference on Learning Representations*, 2020. 1, 3

[5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting Feature Prediction for Learning Visual Representations from Video. *arXiv preprint arXiv:2404.08471*, 2024. 2

[6] Jacob Beck, Risto Vuorio, Evan Zheran Liu, Zheng Xiong, Luisa Zintgraf, Chelsea Finn, and Shimon Whiteson. A Tutorial on Meta-Reinforcement Learning. *Foundations and Trends in Machine Learning*, 18(2–3):224–384, 2025. 2

[7] Carolin Benjamins, Theresa Eimer, Frederik Schubert, Aditya Mohan, Sebastian Döhler, André Biedenkapp, Bodo Rosenhahn, Frank Hutter, and Marius Lindauer. Contextualize Me – The Case for Context in Reinforcement Learning. *Transactions on Machine Learning Research*, 2023. 3

[8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, 2020. 2

[9] Zhenfang Chen, Kexin Yi, Yunzhu Li, Mingyu Ding, Antonio Torralba, Joshua B. Tenenbaum, and Chuang Gan. ComPhy: Compositional Physical Reasoning of Objects and Events from Videos. In *International Conference on Learning Representations*, 2022. 3

[10] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin CM Burchfiel, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 3

[11] Karl Cobbe, Chris Hesse, Jacob Hilton, and John Schulman. Leveraging Procedural Generation to Benchmark Reinforcement Learning. In *Proceedings of the 37th International Conference on Machine Learning*, pages 2048–2056. PMLR, 2020. 3

[12] Pieter-Tjerk de Boer, Dirk P. Kroese, Shie Mannor, and Reuven Y. Rubinstein. A Tutorial on the Cross-Entropy Method. *Annals of Operations Research*, 134(1):19–67, 2005. 6

[13] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. A Survey on In-context Learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1107–1128. Association for Computational Linguistics, 2024. 2

[14] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL^2: Fast Reinforcement Learning via Slow Reinforcement Learning. *arXiv preprint arXiv:1611.02779*, 2016. 2

[15] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993. 12

[16] Fabio Ferreira, Moreno Schlageter, Raghu Rajan, André Biedenkapp, and Frank Hutter. One-shot World Models Using a Transformer Trained on a Synthetic Prior. *arXiv preprint arXiv:2409.14084*, 2024. 2

[17] C. A. Field and A. H. Welsh. Bootstrapping Clustered Data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007. 12

[18] Tilmann Gneiting. Making and Evaluating Point Forecasts. *Journal of the American Statistical Association*, 106(494): 746–762, 2011. 5

[19] David Ha and Jürgen Schmidhuber. Recurrent World Models Facilitate Policy Evolution. In *Advances in Neural Information Processing Systems*, 2018. 1, 2

[20] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning Latent Dynamics for Planning from Pixels. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2555–2565. PMLR, 2019. 1, 2

[21] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. In *International Conference on Learning Representations*, 2020. 1, 2

[22] Steeven Janny, Fabien Baradel, Natalia Neverova, Madiha Nadri, Greg Mori, and Christian Wolf. Filtered-CoPhy: Unsupervised Learning of Counterfactual Physics in Pixel Space. In *International Conference on Learning Representations*, 2022. 3

[23] Yann LeCun. A Path Towards Autonomous Machine Intelligence, 2022. Position paper. 2

[24] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. *arXiv preprint arXiv:2603.19312*, 2026. 1, 2, 6

[25] Allan H. Murphy. Skill Scores Based on the Mean Square Error and Their Relationships to the Correlation Coefficient. *Monthly Weather Review*, 116(12):2417–2424, 1988. 5

[26] Alexander Nikulin, Vladislav Kurenkov, Ilya Zisman, Artem Agarkov, Viacheslav Sinii, and Sergey Kolesnikov. XLand-MiniGrid: Scalable Meta-Reinforcement Learning Environments in JAX. In *Advances in Neural Information Processing Systems*, 2024. 3

[27] Alexander Nikulin, Ilya Zisman, Alexey Zemtsov, and Vladislav Kurenkov. XLand-100B: A Large-Scale Multi-Task Dataset for In-Context Reinforcement Learning. In *International Conference on Learning Representations*, 2025. 3

[28] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking Offline Goal-Conditioned RL. In *International Conference on Learning Representations*, 2025. 3

[29] Kate Rakelly, Aurick Zhou, Chelsea Finn, Sergey Levine, and Deirdre Quillen. Efficient Off-Policy Meta-Reinforcement Learning via Probabilistic Context Variables. In *Proceedings of the 36th International Conference on Machine Learning*, pages 5331–5340. PMLR, 2019. 2

[30] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models. *arXiv preprint arXiv:2502.14819*, 2025. 1, 2, 6

[31] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. DeepMind Control Suite. *arXiv preprint arXiv:1801.00690*, 2018. 3

[32] Hsiao-Yu Tung, Mingyu Ding, Zhenfang Chen, Daniel M. Bear, Chuang Gan, Joshua B. Tenenbaum, Daniel L. K. Yamins, Judith Fan, and Kevin A. Smith. Physion++: Evaluating Physical Scene Understanding that Requires Online Inference of Different Physical Properties. In *Advances in Neural Information Processing Systems*, 2023. 1, 3

[33] Fan Wang, Zhiyuan Chen, Yuxuan Zhong, Sunjian Zheng, Pengtao Shao, Bo Yu, Shaoshan Liu, Jianan Wang, Ning Ding, Yang Cao, and Yu Kang. Context and Diversity Matter: The Emergence of In-Context Learning in World Models. In *International Conference on Learning Representations*, 2026. 1, 2, 5

[34] Siyin Wang, Junhao Shi, Senyu Fei, Zhaoyang Fu, Li Ji, Jingjing Gong, and Xipeng Qiu. In-Context World Modeling for Robotic Control. *arXiv preprint arXiv:2606.26025*, 2026. 2

[35] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. In *Proceedings of the Conference on Robot Learning*, pages 1094–1100. PMLR, 2020. 3

[36] Haochen Zhang, Jiaheng Guo, Zhen Xu, Zachary Plotkin, Nicholas Konz, Zhen Tan, and Tianlong Chen. On the Divergence of Accuracy and Mechanism Consistency in Time Series World Models. *arXiv preprint arXiv:2610.01842*, 2026. 3

[37] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 79115–79135. PMLR, 2025. 2, 3, 6

A. Input Sampling

Observations are typically sampled every five simulator steps. The actions executed between consecutive observations are included in the interaction history.

B. Trajectory Construction and Pairing

Continuous trajectories. Each sample follows a continuous simulator trajectory from its history through its future, without intermediate resets or state overrides. Identical current observations may still conceal differences in velocity or contact state, so we perform task-specific checks of physical-state matching at the start of prediction. For action delay, we retain the actual queue of pending actions.

Starting positions. For tasks in which the two dynamics conditions cannot naturally share a starting position, we exchange their starting positions during generation and verify that the two conditions have identical sets of starting images. This check reduces the possibility that the starting image alone reveals the condition.

Predictions without history. Consider a complete, balanced pair with distinguishable encoded targets. A deterministic model that ignores history receives the same current observation and future action sequence for both samples and therefore produces the same prediction. This prediction cannot be strictly closer to each sample's own target than to the other target. Consequently, such a model scores at most 50% on the paired future-selection metric in Eq. 2; ties count as failures.

C. Action Delay Response Groups

The delay data retain all 11 delays from 0 to 10 simulator steps. At the five-step horizon used for the main one-step score, delays of 5–10 produce the same stationary response and are grouped together. This score therefore uses six distinguishable response groups: 0, 1, 2, 3, 4, and 5–10.

D. Dataset Statistics and Splits

D.1. Dataset Sizes

Table 4 lists the task data used in the training and evaluation comparisons. Training sizes exclude the original-environment data used for mixed training. A pair contains two trajectories with different dynamics conditions; a trajectory count refers to recorded episodes, not to the number of training windows sampled from them. An evaluation case fixes the current observation and future action sequence and can include histories and targets from several dynamics conditions.

The action delay counts refer to the dataset covering all delays from 0 to 10 simulator steps. Its Training split contains 5,120 base cases, each simulated under all 11 delays, giving 56,320 trajectories. Development and Test each contain 300 base cases, again evaluated under all 11 delays. Door Training contains 7,680 trajectories for each of the passable and blocked rules.

Several examples can share a source episode: pusher displacement can use multiple action windows from one episode, and forward and mirrored damping examples can share a source. The 10,000 gripper carry training pairs use 10,000 distinct source episodes.

D.2. Speed Values and Door Positions

Table 5 lists the speed values used for Training and evaluation. Development and Test use the same evaluation speeds but different interaction trajectories. Each split contains 300 cases for each of the 14 evaluated speeds, or 4,200 cases in total before comparing histories from different speeds. The main task results use the unseen speeds within the training range.

Door position is specified by an integer coordinate along the wall. Test uses the 42 positions $30 + 4k$, where $k = 0, \ldots, 41$. Development uses the 16 positions 65, 77, 79, 87, 112, 115, 131, 152, 167, 169, 171, 172, 175, 180, 183, 185. Training uses the integers from 36 through 188 after excluding all Development and Test positions and the additional unused positions 40, 127, and 132, leaving 96 positions. The three position sets are disjoint. Both passable and blocked rules are used across the splits; the test distinction is the door position, not a new rule.

D.3. Split Checks and Model Inputs

Split checks use source episode identifiers, trajectory template identifiers, or observation hashes, as appropriate for each task. Speed Development uses independently generated initial configurations and excludes initial states and current observations that duplicate Test cases. For Door, we also check the position sets listed above for overlap.

Hidden dynamics values, simulator states, and pair identifiers are retained for data generation, validation, and scoring. They are not passed to the prediction model, which receives only the visual observations and actions defined in Sec. 3.1.

E. Evaluation Details

Paired tasks. The strict comparisons in Eq. 2 do not count ties as correct. The two comparisons within a pair are dependent. Appendix B explains the upper bound for a deterministic model that ignores history.

Speed. For each target speed, accuracy is the proportion of cases in which matching history gives strictly lower pre-

Table 4. Dataset sizes and condition coverage.

Task	Training	Development	Test
Speed	16,384 trajectories; 32 speeds	300 cases per speed	300 cases per speed
Pusher displacement	32,768 pairs	256 pairs	256 pairs
Arm mass	32,768 pairs	256 pairs	256 pairs
Action delay	56,320 trajectories; 11 delays	300 cases $\times$ 11 delays	300 cases $\times$ 11 delays
Contact friction	32,768 pairs	256 pairs	256 pairs
Motion damping	32,768 pairs	256 pairs	256 pairs
Gripper carry rule	10,000 pairs	256 pairs	256 pairs
Door traversability	15,360 trajectories; 96 door positions	300 cases; 16 door positions	300 cases; 42 door positions
Portal exit	32,768 pairs	256 pairs	256 pairs

Table 5. Speed values in the benchmark. The four evaluation groups are used in both Development and Test. Interpolation values lie within the Training range but are absent from Training.

Setting	Speed values
Training	2.6, 2.7, 2.8, 2.9, 3.1, 3.3, 3.5, 3.7, 3.8, 3.9, 4.1, 4.2, 4.4, 4.5, 4.7, 5.0, 5.1, 5.3, 5.5, 5.7, 5.9, 6.1, 6.2, 6.3, 6.6, 6.7, 6.8, 7.0, 7.2, 7.3, 7.8, 7.9
Seen speeds	3.1, 5.1, 7.0
Interpolation	3.4, 4.8, 6.9
Lower extrapolation	1.75, 1.95, 2.15, 2.35
Upper extrapolation	8.25, 8.75, 9.5, 10.25

diction error than every alternative history. We then average accuracy equally across target speeds. Ties do not count as correct: if all histories produce the same prediction, accuracy is zero. The main results use the interpolation speeds listed in Appendix D.

Action delay. We select the candidate true future with the lowest mean squared error, resolving ties in favor of the smaller delay. The prediction is correct when the selected delay and the true delay belong to the same response group. Within each case, we first average correctness over delays in each group, then average equally across the six groups defined in Appendix C. The final accuracy is the mean over cases.

Door traversability. For a prediction made with matching history, we compare the mean squared errors to the passage and blockage futures. The prediction is correct only if the error to the true future is strictly smaller than the error to the alternative future. Accuracy is averaged over cases and the two rules; ties do not count as correct.

Multi-step prediction horizons. Each prediction step advances five simulator steps. We measure prediction errors at 5, 10, 15, 20, and 25 simulator steps after the current observation.

Mean-future reference. The reference in Eq. 3 is used only for scoring and is never provided to the model. The ratio is defined only when $\sum_q B_q > 0$. When conditions share the current observation and future actions and are equally weighted, a deterministic model that ignores history cannot achieve $E/B < 1$: it must make the same prediction across conditions, and the mean-future reference minimizes the error of such a prediction.

Response error settings. We specify the prediction horizon for each NRE analysis. The ratio in Eq. 5 is defined only when $\sum_i \|\Delta z_i\|^2 > 0$.

Uncertainty estimates. For multi-step errors, we use cluster bootstrap resampling [15, 17] at the level of source trajectories or scenes, retaining related samples together. These intervals describe scene-sampling uncertainty for the evaluated checkpoints, not variability across training runs.

Interpreting multiple metrics. The metrics describe different aspects of the same predictions and are not independent pieces of evidence.

F. Scoring Controls

We validate the paired scorer using constructed predictions on 256 Development pusher-displacement scenes, separately in the target representations of LeWM Original and Original → Mixed (Frozen Encoder). The queries use a nonzero action candidate and the five-step prediction horizon; the predictor is not run for this check. Both representations give the results below. With $m = (z_A + z_B)/2$, even predictions $m + 0.001(z_k - m)$ select the correct future while recovering only a tiny fraction of the true response. This control explains why we report response error alongside task accuracy. It verifies the scorer on these queries, not physical calibration across all representations.

Constructed prediction	Accuracy (%)	History (%)	NRE
True target for each condition	100	100	0
Same target z_A for both conditions	50	0	1
Same midpoint m for both conditions	0	0	1
Swapped true targets	0	0	4
$m + 0.001(z_k - m)$ for condition k	100	100	0.998001

G. Training and Diagnostic Settings

Optimization. Each training stage runs for 10 epochs on eight GPUs, with 128 windows per device and no gradient accumulation, giving a global batch size of 1,024. LeWM and PLDM use AdamW with learning rate 5×10^{-5} and weight decay 0.001. DINO-WM uses learning rate 5×10^{-4} and zero weight decay; its pretrained DINOv2-small visual encoder remains frozen. The models retain their respective prediction and regularization losses. The Original → Mixed schemes load model weights but reset optimizer state. Encoder freezing fixes both parameters and training-state buffers; it does not freeze the projector or action encoder.

Input and sampling. The standard setup uses three observation frames, five simulator steps between frames, and a one-frame prediction target. Action delay mixed training instead uses seven-frame history. The original-environment data are the TwoRoom, PushT, Reacher, and Cube datasets released with LeWM; each task uses only the dataset for its environment. Only the ContextWorld Training split is mixed into training. For component lengths N_{orig} and N_{CW}, the mixture has a nominal epoch length of $2\max(N_{\mathrm{orig}}, N_{\mathrm{CW}})$ windows before partitioning and batching, with equal counts from the two components. The smaller component is sampled repeatedly. Consequently, equal epoch counts do not guarantee equal update counts across data scales, and the two-stage schemes include additional original-environment training.

Prediction diagnostics. The main task comparison uses Test. The multi-step and history diagnostics below use Development, with the same checkpoint identities where the input protocol is comparable. Multi-step evaluation covers 2,436 source scenes across the nine tasks and predicts at 5, 10, 15, 20, and 25 simulator steps using only the initial observed history. The candidate actions and multi-step targets extend the one-step task queries. Each checkpoint is scored separately, and matching training repetitions are then averaged equally. Intervals use 4,000 source-cluster bootstrap samples; all conditions, actions, and times from a source remain together. They condition on the evaluated checkpoints and do not estimate variation from retraining the model. Original action-delay checkpoints do not have the native seven-frame interface and are excluded from this comparison. DINO-WM has no jointly updated visual-encoder scheme; its comparable Original → Mixed results cover Speed, damping, and gripper carry.

Planning evaluation. CEM results measure success in the original environments, with no hidden-dynamics history intervention. Most checkpoint evaluations use six evaluation seeds with 50 episodes each; some original LeWM and PLDM references use three sets of 100 episodes or an aggregated 300-episode result. Evaluation repetitions are pooled within each checkpoint, then training repetitions are averaged equally. A single original-environment checkpoint supplies the reference for all tasks from that environment; these repeated entries are not independent experiments.

Training-query analysis. The friction and damping comparisons use 32,768 training pairs, identical original-environment initialization within each task, and the same 50/50 mixture, batch size, and duration. We evaluate 512 fixed training pairs and 256 Development pairs per task at epochs 5 and 10, predicting the next observation five simulator steps ahead. Loss summaries use the logged mixed-training batches, approximately 50–70 records per epoch, rather than every training batch.

Checkpoints and logs are aligned by update step. The curves show recorded means and batch interquartile ranges; loss medians give the same monotonic direction for friction joint training from epochs 5 to 10. NRE is evaluated in each checkpoint's target representation and is not a common physical error across checkpoints. For damping with the encoder frozen, prediction loss changes by only -0.39% between these endpoints and fluctuates in between; we describe it as approximately constant.

H. Additional Results

The following tables separate prediction diagnostics from original-environment planning. History measures strict error reduction from matching history on the task queries; NRE measures the conditional response on those queries. E/B uses the extended multi-step action panels and includes errors shared by both conditions. Because the query sets and prediction horizons differ, these columns should not be treated as measurements of the same event. All latent errors are computed in the checkpoint's own target representation.

Table 6. Development prediction diagnostics and original-environment CEM. Entries average the available training repetitions; E/B intervals are 95% source-bootstrap intervals. History and CEM are percentages. Scheme names match the main text.

Task	Model	Scheme	History ↑	NRE ↓	E/B ↓	E/B interval	CEM ↑
Speed	LeWM	Original	59.15	0.896	1.498	[1.478, 1.517]	92.44
Speed	LeWM	Mixed	98.00	0.042	0.363	[0.345, 0.381]	97.44
Speed	LeWM	Original → Mixed (Joint)	96.67	0.055	0.406	[0.384, 0.429]	97.00
Speed	LeWM	Original → Mixed (Frozen Encoder)	98.56	0.030	0.144	[0.132, 0.156]	95.00
Speed	PLDM	Original	48.48	0.925	3.377	[3.309, 3.445]	90.56
Speed	PLDM	Mixed	97.19	0.084	0.520	[0.498, 0.544]	96.89
Speed	PLDM	Original → Mixed (Joint)	98.89	0.046	0.353	[0.327, 0.379]	96.33
Speed	PLDM	Original → Mixed (Frozen Encoder)	95.56	0.065	0.302	[0.285, 0.319]	98.00
Speed	DINO-WM	Original	58.56	0.928	1.973	[1.954, 1.991]	100.00
Speed	DINO-WM	Mixed	56.56	0.967	2.101	[2.070, 2.133]	99.89
Speed	DINO-WM	Original → Mixed (Frozen Encoder)	86.00	0.411	0.886	[0.847, 0.927]	100.00
Pusher displacement	LeWM	Original	34.05	1.015	2.378	[2.284, 2.481]	82.22
Pusher displacement	LeWM	Mixed	73.63	0.981	3.572	[3.421, 3.741]	78.67
Pusher displacement	LeWM	Original → Mixed (Joint)	97.46	0.222	2.504	[2.425, 2.586]	86.00
Pusher displacement	LeWM	Original → Mixed (Frozen Encoder)	98.05	0.144	1.464	[1.396, 1.537]	73.67
Pusher displacement	PLDM	Original	15.89	1.048	2.471	[2.395, 2.552]	75.67
Pusher displacement	PLDM	Mixed	66.60	0.896	6.533	[6.241, 6.847]	61.33
Pusher displacement	PLDM	Original → Mixed (Joint)	73.83	0.798	3.811	[3.664, 3.968]	72.00
Pusher displacement	PLDM	Original → Mixed (Frozen Encoder)	99.61	0.115	1.445	[1.388, 1.506]	59.33
Pusher displacement	DINO-WM	Original	14.65	1.113	3.300	[3.166, 3.446]	76.67
Pusher displacement	DINO-WM	Mixed	98.63	0.432	1.996	[1.941, 2.055]	76.67
Arm mass	LeWM	Original	50.98	0.977	687.381	[571.339, 816.332]	55.89

Task	Model	Scheme	History ↑	NRE ↓	E/B ↓	E/B interval	CEM ↑
Arm mass	LeWM	Mixed	91.60	0.048	51.291	[45.684, 56.751]	58.00
Arm mass	LeWM	Original → Mixed (Joint)	91.80	0.039	47.152	[41.802, 52.374]	59.00
Arm mass	LeWM	Original → Mixed (Frozen Encoder)	88.28	0.338	31.814	[27.581, 35.547]	59.67
Arm mass	PLDM	Original	66.80	0.922	18.594	[16.630, 20.929]	69.67
Arm mass	PLDM	Mixed	92.19	0.033	13.617	[12.780, 14.631]	51.67
Arm mass	PLDM	Original → Mixed (Joint)	91.02	0.077	44.421	[38.659, 50.208]	58.67
Arm mass	PLDM	Original → Mixed (Frozen Encoder)	92.38	0.211	16.644	[15.067, 18.031]	89.00
Arm mass	DINO-WM	Original	59.38	0.980	48.246	[45.632, 51.198]	90.67
Arm mass	DINO-WM	Mixed	89.26	0.658	14.246	[13.770, 14.655]	75.00
Action delay	LeWM	Mixed	99.00	0.012	1.462	[1.439, 1.487]	87.33
Action delay	PLDM	Mixed	99.99	0.007	1.807	[1.795, 1.820]	90.89
Action delay	DINO-WM	Mixed	21.14	1.000	65.764	[64.835, 66.676]	33.56
Contact friction	LeWM	Original	50.59	1.001	2.753	[2.680, 2.834]	82.22
Contact friction	LeWM	Mixed	50.00	0.999	2.693	[2.642, 2.747]	76.67
Contact friction	LeWM	Original → Mixed (Joint)	57.23	0.999	2.714	[2.674, 2.758]	86.00
Contact friction	LeWM	Original → Mixed (Frozen Encoder)	92.77	0.448	2.891	[2.833, 2.953]	83.33
Contact friction	PLDM	Original	49.28	1.012	3.869	[3.753, 3.992]	75.67
Contact friction	PLDM	Mixed	50.00	1.001	6.269	[5.975, 6.602]	65.67
Contact friction	PLDM	Original → Mixed (Joint)	51.37	1.004	7.086	[6.828, 7.366]	73.33
Contact friction	PLDM	Original → Mixed (Frozen Encoder)	85.94	0.604	4.499	[4.391, 4.618]	79.67
Contact friction	DINO-WM	Original	48.63	1.019	6.109	[5.850, 6.372]	76.67
Contact friction	DINO-WM	Mixed	93.16	0.684	3.673	[3.603, 3.744]	75.67
Motion damping	LeWM	Original	49.09	0.995	5.308	[5.042, 5.616]	82.22
Motion damping	LeWM	Mixed	49.61	1.002	291.614	[253.071, 335.959]	78.00
Motion damping	LeWM	Original → Mixed (Joint)	50.20	1.001	95.787	[83.792, 110.333]	88.33
Motion damping	LeWM	Original → Mixed (Frozen Encoder)	58.79	0.997	8.202	[7.657, 8.832]	84.33
Motion damping	PLDM	Original	51.69	0.957	5.973	[5.668, 6.318]	75.67
Motion damping	PLDM	Mixed	58.59	1.521	25.424	[22.201, 29.444]	61.00
Motion damping	PLDM	Original → Mixed (Joint)	79.10	0.221	8.247	[7.957, 8.547]	70.00
Motion damping	PLDM	Original → Mixed (Frozen Encoder)	62.11	0.882	5.553	[5.276, 5.871]	78.33

Task	Model	Scheme	History ↑	NRE ↓	E/B ↓	E/B interval	CEM ↑
Motion damping	DINO-WM	Original	52.34	1.001	8.465	[8.122, 8.825]	76.67
Motion damping	DINO-WM	Mixed	66.21	1.021	7.273	[7.077, 7.482]	76.67
Motion damping	DINO-WM	Original → Mixed (Frozen Encoder)	65.62	1.083	6.927	[6.731, 7.135]	76.00
Gripper carry rule	LeWM	Original	74.80	0.911	11.678	[10.728, 12.783]	65.44
Gripper carry rule	LeWM	Mixed	49.61	1.001	198.472	[166.622, 241.013]	61.33
Gripper carry rule	LeWM	Original → Mixed (Joint)	52.34	0.998	13.085	[11.613, 14.946]	71.00
Gripper carry rule	LeWM	Original → Mixed (Frozen Encoder)	78.12	0.320	11.937	[11.096, 12.883]	67.00
Gripper carry rule	PLDM	Original	57.62	0.923	15.350	[12.126, 22.152]	53.44
Gripper carry rule	PLDM	Mixed	52.54	0.985	147.591	[121.468, 184.796]	55.00
Gripper carry rule	PLDM	Original → Mixed (Joint)	54.49	0.985	26.553	[24.128, 29.592]	56.67
Gripper carry rule	PLDM	Original → Mixed (Frozen Encoder)	58.01	0.594	29.969	[21.538, 46.388]	56.67
Gripper carry rule	DINO-WM	Original	49.80	1.001	42.857	[41.696, 44.068]	72.67
Gripper carry rule	DINO-WM	Mixed	80.66	0.890	11.789	[11.454, 12.141]	71.67
Gripper carry rule	DINO-WM	Original → Mixed (Frozen Encoder)	82.03	0.874	11.747	[11.417, 12.082]	73.67
Door traversability	LeWM	Original	45.72	0.997	4.676	[4.568, 4.784]	92.44
Door traversability	LeWM	Mixed	100.00	0.045	2.981	[2.938, 3.024]	95.11
Door traversability	LeWM	Original → Mixed (Joint)	100.00	0.047	2.427	[2.378, 2.478]	97.67
Door traversability	LeWM	Original → Mixed (Frozen Encoder)	100.00	0.036	2.442	[2.400, 2.483]	94.00
Door traversability	PLDM	Original	52.22	0.997	7.520	[7.323, 7.718]	90.56
Door traversability	PLDM	Mixed	100.00	0.021	2.404	[2.384, 2.424]	95.78
Door traversability	PLDM	Original → Mixed (Joint)	100.00	0.027	2.349	[2.302, 2.397]	99.33
Door traversability	PLDM	Original → Mixed (Frozen Encoder)	100.00	0.049	2.455	[2.419, 2.491]	98.67
Door traversability	DINO-WM	Original	61.00	0.986	4.233	[4.143, 4.321]	100.00
Door traversability	DINO-WM	Mixed	100.00	0.223	1.329	[1.303, 1.354]	97.56
Portal exit	LeWM	Original	51.82	0.990	822.254	[587.576, 1303.713]	92.44
Portal exit	LeWM	Mixed	77.54	0.918	58.552	[47.450, 74.465]	98.33

Task	Model	Scheme	History ↑	NRE ↓	E/B ↓	E/B interval	CEM ↑
Portal exit	LeWM	Original → Mixed (Joint)	66.21	0.919	61.538	[49.099, 81.373]	95.67
Portal exit	LeWM	Original → Mixed (Frozen Encoder)	93.16	0.253	13.286	[10.625, 17.147]	89.33
Portal exit	PLDM	Original	51.63	0.996	1056.976	[789.611, 1513.078]	90.56
Portal exit	PLDM	Mixed	78.71	0.121	37.556	[35.991, 39.195]	97.00
Portal exit	PLDM	Original → Mixed (Joint)	62.11	0.694	21.881	[20.065, 24.077]	98.67
Portal exit	PLDM	Original → Mixed (Frozen Encoder)	98.24	0.203	10.485	[9.219, 12.101]	98.67
Portal exit	DINO-WM	Original	54.69	0.995	41.991	[40.909, 43.144]	100.00
Portal exit	DINO-WM	Mixed	100.00	0.143	8.092	[8.005, 8.181]	100.00

H.1. Data Scale

These comparisons use Development and hold training epochs fixed. The sample counts describe pairs, not necessarily independent source episodes. Pusher displacement uses 2,048, 8,192, and 32,768 pairs; friction and damping compare 8,192 with 32,768; mass and portal exit compare 2,048 with 32,768. Gripper carry expands from 2,048 to 10,000 source episodes. Portal expansion also broadens initial distances and action magnitudes, so its comparison changes coverage as well as size.

Table 7. Data-scale comparisons on Development. n counts training repetitions contributing to task accuracy and NRE. Planning repetitions can differ and are not counted as additional training runs.

Task	Model	Scheme	Pairs	n	Accuracy (%)	NRE
Pusher displacement	LeWM	Mixed	2,048	3	50.00	1.000
Pusher displacement	LeWM	Mixed	32,768	1	50.78	0.981
Pusher displacement	LeWM	Original → Mixed (Joint)	2,048	1	55.66	0.810
Pusher displacement	LeWM	Original → Mixed (Joint)	8,192	1	82.42	0.390
Pusher displacement	LeWM	Original → Mixed (Joint)	32,768	1	90.82	0.222
Pusher displacement	PLDM	Mixed	2,048	3	50.07	1.003
Pusher displacement	PLDM	Mixed	32,768	1	56.64	0.896
Pusher displacement	DINO-WM	Mixed	2,048	3	89.19	0.583
Pusher displacement	DINO-WM	Mixed	32,768	1	96.09	0.432
Arm mass	LeWM	Mixed	2,048	3	85.35	0.132
Arm mass	LeWM	Mixed	32,768	1	87.11	0.048
Arm mass	PLDM	Mixed	2,048	3	52.60	1.054
Arm mass	PLDM	Mixed	32,768	1	91.21	0.033
Arm mass	DINO-WM	Mixed	2,048	3	51.37	0.999
Arm mass	DINO-WM	Mixed	32,768	1	80.47	0.658
Contact friction	LeWM	Mixed	8,192	3	50.07	1.000
Contact friction	LeWM	Mixed	32,768	1	50.00	0.999
Contact friction	PLDM	Mixed	8,192	3	50.13	1.001
Contact friction	PLDM	Mixed	32,768	1	50.20	1.001
Contact friction	DINO-WM	Mixed	8,192	3	49.61	1.025
Contact friction	DINO-WM	Mixed	32,768	1	81.84	0.684

Task	Model	Scheme	Pairs	n	Accuracy (%)	NRE
Motion damping	LeWM	Mixed	8,192	3	50.07	0.999
Motion damping	LeWM	Mixed	32,768	1	49.80	1.002
Motion damping	PLDM	Mixed	8,192	3	51.89	2.122
Motion damping	PLDM	Mixed	32,768	1	58.40	1.521
Motion damping	DINO-WM	Mixed	8,192	3	50.13	1.014
Motion damping	DINO-WM	Mixed	32,768	1	57.42	1.021
Gripper carry rule	LeWM	Mixed	2,048	3	49.93	1.001
Gripper carry rule	LeWM	Mixed	10,000	1	50.00	1.001
Gripper carry rule	PLDM	Mixed	2,048	3	50.00	0.996
Gripper carry rule	PLDM	Mixed	10,000	1	50.00	0.985
Gripper carry rule	DINO-WM	Mixed	2,048	3	50.33	0.995
Gripper carry rule	DINO-WM	Mixed	10,000	1	61.33	0.890
Portal exit	LeWM	Mixed	2,048	3	50.91	0.925
Portal exit	LeWM	Mixed	32,768	1	51.95	0.918
Portal exit	PLDM	Mixed	2,048	3	69.53	0.155
Portal exit	PLDM	Mixed	32,768	1	73.63	0.121
Portal exit	DINO-WM	Mixed	2,048	3	99.41	0.554
Portal exit	DINO-WM	Mixed	32,768	1	100.00	0.143

H.2. Model Capacity

Both model sizes use the same 32,768-pair datasets, training seed, Mixed initialization, and 10-epoch recipe. The larger model changes the visual backbone from ViT-Tiny to ViT-Base, the representation width from 192 to 768, and predictor depth from six to nine layers. This is a joint expansion of the encoder and predictor, not a predictor-only intervention.

		Accuracy (%)		NRE		CEM (%)	
Task	Model	18M	183M	18M	183M	18M	183M
Contact friction	LeWM	50.00	49.80	0.999	1.000	76.67	74.00
Motion damping	LeWM	49.80	50.00	1.002	1.000	78.00	71.33
Contact friction	PLDM	50.20	49.80	1.001	1.107	65.67	47.33
Motion damping	PLDM	58.40	50.39	1.521	1.536	61.00	50.33

The four endpoint comparisons do not show a consistent improvement from increasing capacity under this recipe. Only measured checkpoints are used in the training-curve analysis: epochs 1–10 for the large LeWM models and epochs 5 and 10 for the other models. There is no shared trajectory of accurate training responses followed by worsening Development response error across these runs. This does not exclude earlier overfitting where checkpoints are unavailable, or gains from retuning the larger models.